EAC Algorithm (raw form) for Eliminating Indirect Left Recursion

Algorithm: Eliminate Left Recursion

```
1. Order the non-terminals as (A_1, A_2, ..., A_n)
2. for ( i = 1 ) to ( n ) do
    2.1 for ( j = 1 ) to ( i-1 ) do
        Replace each production
        (A_i → A_j α)
        by the productions
        (A_i → β α)
        for each (A_j → β)

    2.2 Eliminate direct left recursion on (A_i)
```

Line 1

"Order the non-terminals as $(A_1, A_2, \ldots, A_n)$"

What this really means:

- Take the existing non-terminals in the grammar
- Put them in *any fixed order*
- Attach index labels so the algorithm can refer to "earlier" vs "later"

Why this exists:

- Prevents infinite substitution loops
- Makes progress measurable
- Has *nothing* to do with grammar meaning

Line 2

"for (i = 1) to (n) do"

Meaning:

- Process non-terminals one at a time
- When you start $(A_i), all(A_1 \ldots A_{i-1})$ are already cleaned

Invariant (important):

> After finishing iteration (i), **no production of any** (A_k) **with** $(k \leq i)$ **starts with** (A_m) **where** $(m \leq k)$

This invariant is why the algorithm terminates.

Line 2.1

"for (j = 1) to (i-1) do"

Meaning:

- Look at *earlier* non-terminals only
- Never substitute "forward"

Why:

- Earlier non-terminals are already safe
- Forward substitution could reintroduce recursion

Line 2.1 (core step)

"Replace each production $(A_i ::= A_j\alpha)$**"**

What you are checking:

- Scan *all RHSs* of (A_i)
- Look for RHSs that *start* with (A_j)

This is **indirect** left recursion detection.

Line 2.1 (replacement rule)

"by $(A_i ::= \beta\alpha)$ **for each** $(A_j ::= \beta)$**"**

This is substitution (not merging).

You are saying:

> "If expanding (A_i) starts by calling (A_j), inline all ways (A_j) can expand."

So if:

$A_j ::= \beta_1|\beta_2$

Then:

$A_i ::= A_j\alpha$

becomes:

$A_i ::= \beta_1\alpha|\beta_2\alpha$

Why this works:

- Makes the hidden recursion explicit
- Moves dependency to the front

- Preserves the language

Line 2.1 — subtle but crucial detail

You must do this:

- for **every j < i**
- for **every matching RHS**
- before moving to direct elimination

This is what exposes *all* indirect recursion.

Line 2.2

"Eliminate direct left recursion on (A_i)**"**

Now — and only now — you do the standard transform:

If you have:

$A_i ::= A_i\alpha_1|A_i\alpha_2|...|\beta_1|\beta_2$

(where each β does **not** start with (A_i))

Transform to:

$A_i ::= \beta_1 A_i'|\beta_2 A_i'$ $A_i' ::= \alpha_1 A_i'|\alpha_2 A_i'|\epsilon$

Why this is safe now:

- No RHS starts with earlier non-terminals
- Only self-recursion remains
- This step cannot reintroduce indirect recursion

Big picture invariant (this ties it all together)

After finishing iteration (i):

- (A_i) has **no left recursion**
- (A_i) does not start with any (A_j) for $(j \leq i)$
- All recursion has been pushed *rightward*

How to say this out loud (teaching version)

> "For each nonterminal, first expand away anything that starts with an earlier nonterminal. Once that's done, the only possible left recursion is direct — and we know how to remove that."

One-line mental model

"Inline earlier symbols until recursion is visible; then eliminate it."